

The Readability Spectrum: Patterns, Issues, and Prompt Effects in LLM-Generated Code

Hengzhi Ye
hzye@stu.pku.edu.cn
Peking university
Beijing, Beijing, China

Weiwei Xu
xuww@stu.pku.edu.cn
Peking university
Beijing, Beijing, China

Fengyuan Ran
rfy_reflow@whu.edu.cn
Wuhan University
Wuhan, Hubei, China

Minghui Zhou
zhmh@pku.edu.cn
Peking university
Beijing, Beijing, China

Abstract

As Large Language Models (LLMs) are transforming software development, the functional quality of generated code has become a central focus, leaving readability, one of critical non-functional attributes, understudied. Given that LLM-generated code still needs human review before adoption, it is important to understand its readability especially compared with human-written code and the role of prompt design in shaping it. We therefore set out to conduct a systematic investigation into the code readability of LLM-generated code. To systematically quantify code readability, We establish a comprehensive readability model that synthesizes textual, structural, program, and visual features of code. Based on the model, we evaluate the readability of code generated by the mainstream LLMs under 5,869 scenarios extracted from large code base including World of Code (WoC) and LeetCode. We find that current LLMs produce code with overall readability comparable to human-written code, but displaying distinct readability issue patterns. We further examine how different prompt dimensions affect the readability of LLM-generated code, and find that function signatures, constraints and style descriptions emerge as the most influential factors, while the overall impact of prompt design remains limited. Our findings indicate that, on one hand, LLM-generated code is at least comparable to human-written code in readability, validating its potential for systematic integration into software workflows from a non-functional perspective; on the other hand, distinct readability issue patterns and limited effectiveness of prompt engineering reveal a latent technical debt, highlighting the need for future research to improve the readability of LLM-generated code and thus ensure long-term maintainability.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

Code readability, code generation, prompt engineering, LLM

1 Introduction

Software development has evolved far beyond the simple act of writing code. Reading and understanding code constitutes the fundamental first step in any maintenance and incremental task [73]. Clear and readable code significantly facilitates comprehension,

reduces maintenance burden, and mitigates technical debt [11, 21]. Consequently, the software engineering community has long advocated for considering code readability during development [25].

The prevalence of Large Language Models (LLMs) has transformed automated programming by reducing developers' efforts in from-scratch development [35, 56]. However, LLM-generated code often suffers from hallucinations, misinterpreted requirements, or outdated knowledge, necessitating rigorous human oversight to mitigate vulnerabilities [53, 102]. While existing research has extensively validated the functional correctness of LLM-generated solutions [17, 18], the readability of LLM-generated code remains underexplored. As human review remains important for verifying AI-generated outputs, unreadable code not only impedes comprehension but may also require more audit effort than manual authoring, thereby undermining the primary benefits of AI assistance [103]. Consequently, a comprehensive evaluation of readability is essential to understanding emerging LLM-integrated development paradigm and providing a foundation for future AI-assisted programming.

To address this knowledge gap, this paper presents a systematic investigation into the readability of LLM-generated code following three core research questions:

- **RQ1:** Does LLM-generated code exhibit higher or lower readability scores compared to human-written code?
- **RQ2:** Does LLM-generated code exhibit distinct patterns of readability issues compared to human-written code?
- **RQ3:** Does prompt engineering influence the readability of LLM-generated code?

We started from establishing a comprehensive readability model by integrating textual, structural, program, and visual features into a unified framework, allowing for a robust assessment of code readability. Based on the model, we first evaluated the readability scores of LLM-generated code in comparison to human-written code across 5,869 scenarios extracted from World of Code (WoC) [57] and LeetCode [1]. To characterize the qualitative nuances of the generated code, the second stage of our analysis investigated whether LLM-generated code exhibits specific readability issue patterns different from human-written code. Furthermore, we explored the potential factors that influence these outcomes, specifically examining how various dimensions of prompt engineering affect readability within the context of single-turn interactions.

We found that the readability of LLM-generated code is at least comparable to that of human-written code, while a critical divergence exists: LLM-generated code exhibits distinct readability issue patterns compared to human-written code, including unnecessary complex structures, low-information comments, and unknown API usage. Further prompt analysis indicated that while function signatures, constraints and style descriptions influence the readability of generated code, its overall impact is constrained and remains insufficient to resolve the readability challenges. Significantly, our findings not only validated the potential for systematic LLM adoption from an overall readability perspective, but also exposed the “hidden technical debt” in AI-assisted programming, offering insights for managing AI-generated code and advancing better LLM-based development assistants.

In summary, this paper provides five main contributions:

- A **scalable benchmark** to compare LLM-generated code and human-written code, consisting of 5,869 standard prompts and corresponding human-implemented code across software engineering practice and programming competitions;
- An **empirical study** showing that LLMs have achieved readability scores at least comparable to those of humans, establishing a quantitative baseline;
- **Identification of AI-specific readability issue patterns**, distinguishing AI-assisted programming technical debt from traditional human coding habits;
- An empirical **quantification of prompt efficacy**, establishing practical design guidelines while revealing the insufficiency of prompting alone to address readability issues;
- A comprehensive **readability assessment framework** that integrates textual, structural, program, and visual features to systematically and quickly quantify code readability.

In Section 2, we review related work. We illustrate the overall methodology in Section 3 and explain the readability model in Section 4. Sections 5 to 7 detail the results for each research question. We discuss implications and limitations in Section 8 and conclude in Section 9.

2 Background and Related Work

2.1 LLM Code Generation

The application of LLMs to code generation substantially enhanced automated programming capabilities. Not only have general LLMs like ChatGPT been widely deployed for code generation [42], but fine-tuned models such as Codex [17] and StarCoder [50] have also emerged to achieve competitive performance across various programming languages. Integrated code agents like Copilot [28] and Cursor [6] further enhance developer productivity. Surveys indicate that Copilot is behind an average of 46% of developers’ code across all programming languages [74], highlighting the profound impact of LLM-generated code on software development.

Researchers have focused on examining whether LLM-generated code is functionally correct, secure, and aligned with developer expectations. For instance, CodeXGLUE [55], a comprehensive evaluation framework comprising 14 tasks including code completion, generation, and translation, established a systematic assessment standard for code generation tasks, while HumanEval

benchmark [17] offered a more targeted approach, utilizing reference solutions and unit tests to precisely quantify the functional correctness of LLM-generated code. Additional evaluation criteria have also examined LLM-generated code in terms of vulnerability risks, privacy protection, and copyright compliance [99, 100]. However, little attention is devoted to the readability of the generated code despite its importance to long-term software lifecycle.

2.2 Code Readability

Since the emergence of programming, code readability has become an implicit code quality criterion. Terms such as “*clean*”, “*good*”, and “*beautiful*” have long served as intuitive descriptors for highly readable code [23, 65]. Despite the early lack of standardized readability metrics, developers have long prioritized aesthetic and readable code, with studies investigating how identifier strategies and comments influence perceived code quality [37, 47].

Buse and Weimer [14, 15] developed the first readability model by mapping 25 structural features, such as indentation and line length, to human-annotated code. Their findings established a positive correlation between readability scores and overall code quality. Subsequent studies refined structural metrics, with Posnett et al. [72] achieving higher accuracy through a simpler three-feature model and Johnson et al. [43] exploring how nesting and looping constructs influenced readability. A significant advancement occurred as Scalabrino et al. [78, 79] integrated textual and semantic features with structural ones to capture code’s dual nature as both logic and language. Most recently, Vitale et al. [88] moved toward personalization by incorporating individual developers’ reading preferences and habits into readability assessments.

Considering practical software engineering contexts, Piantadosi et al. [71] studied readability fluctuations during software evolution, proposing guidelines to prevent readability degradation. Parallel researches have also examined code readability within LLM context. Takerngsaksiri et al. [83] explored the practitioners’ perspectives on code readability in the age of LLMs through a case study, while Pan et al. [67] investigated the influence that readability have on the LLM budget when LLM processing code snippets. Furthermore, Hu et al. [36] investigated LLMs’ performance when interpreting poor-readability code, finding that even LLMs struggle with low-readability code, which mirrors human developers’ difficulties that lead to technical debt accumulation. These findings further emphasizes the detrimental impact of low-readability code in software engineering.

2.3 Prompt Engineering

As an efficient alternative to costly tuning and retraining, prompt engineering has emerged as a primary paradigm for optimizing LLM performance across diverse tasks [59, 93]. Since prompt optimizations directly shape model outputs, they likely influence the readability of the generated code. Consequently, this study investigates the impact of prompting strategies within the context of single-turn interactions and we review related prompt engineering research here.

Leveraging their extensive pre-training, LLMs function as capable few-shot learners through zero-shot instructions or in-context

examples [69, 91]. By activating latent knowledge without resource-intensive fine-tuning, these low-cost and flexible techniques have gained widespread attention among both researchers and users, coalescing into the formalized discipline of prompt engineering.

The adoption of prompt engineering across various domains has established task-specific best practices. Researchers have defined design principles for specialized contexts such as academic writing and business management [13, 27], while practical implementations like “cursor rules” [7] utilize pre-set contexts to optimize code generation. These frameworks improve LLMs’ output by explicitly shaping the input prompt [7, 27, 54], including persona settings, stylistic guidelines, function signature definition, technical contracts, etc. Beyond structural optimization, prompt engineering also incorporates reasoning-oriented methods. Chain-of-Thought (CoT) prompting [92] enhances performance by simulating step-by-step problem decomposition. Furthermore, the development of algorithmic automatization like the Automatic Prompt Engineer (APE) [41] has transitioned the field from heuristic adjustments to a systematic and algorithm-driven discipline.

In summary, current research on LLM code generation primarily emphasizes functional capacity, and similarly, prompt engineering literature has primarily prioritized strategies for enhancing functional performance, leaving the crucial non-functional dimension of readability unexplored. Establishing a readability baseline, identifying specific issue patterns, and evaluating the impact of prompt engineering thus remain critical yet unaddressed knowledge gaps.

3 Methodology

As shown in Figure 1, we employed a mixed-method approach to address our research questions. We started by collecting and processing source data to obtain structured prompt and respective human implementation. Then we prompted LLMs to generate code responses, and compared the readability of LLM-generated code with human-written code. After that, we identified readability issues through manual labeling and thematic analysis, and investigated the impact of different prompt dimensions on generated code readability using various statistical methods.

In the following sections, we detail the approaches of data collection, prompt crafting, prompting and processing, thematic analysis, and statistical analysis.

3.1 Data Collection

To make LLM application scenarios more representable, we selected both real software engineering development contexts and competitive programming problems as task backgrounds. Specifically, we used World of Code (WoC) [57] and LeetCode [1] as data sources. Note that, to ensure the reliability of the human-written baseline, we selected data snapshots prior to 2022. Given that ChatGPT was released in November 2022 [64], before which LLMs were neither widely adopted nor integrated into programming workflows, this temporal boundary justifies the assumption that the collected code is human-written.

WoC is a comprehensive infrastructure for mining version control data across open-source software ecosystem, which aggregates Git objects, including commits, trees and blobs, on platforms like GitHub, Bitbucket, and GitLab [57, 58]. It provides mappings from

project identification to project code, blobs, version control information, and other data for each project entity, facilitating easy querying and management.

We selected version U of WoC (released in October 2021) and used the c2fbb database, which maps each commit to its corresponding commit file name, new blob (post-commit), and old blob (pre-commit), filtering for Python project files¹. To align with Buse and Weimer’s definition of readability [15], we utilized function-level snippets to ensure evaluators have sufficient context and to avoid spanning multiple methods. This granularity provides adequate volume for the reading process while minimizing confounding factors that could distort judgment. We therefore extracted and segmented Python source files into individual function-level entities for analysis. We further divided each snippet into a function header and body: the header, containing the function signature and docstring description, would serve as the prompt to stimulate LLM code generation, while the body, representing the concrete implementation, would serve as human-written baseline. After filtering and selection, we finally obtained 3,000 prompt-baseline pairs.

LeetCode [1] is an online platform that provides programming problems spanning various difficulty levels and topics. It offers a standardized problem set with clear problem descriptions in natural languages, and worldwide users provide a diverse array of human-written code in multiple programming languages [18].

We crawled all accessible problems from LeetCode, including their problem descriptions and code solutions, and then filtered for solutions implemented in Python and released before 2022. Subsequently, we extracted the core function implementation from each code solution to serve as the human-written baseline, with the corresponding problem description serving as the prompt. Note that, given that primary training corpora for LLMs are in English [51], we selected English descriptions to maximize LLM capabilities.

Ultimately, we obtained 2,869 pairs from LeetCode. Combining with WoC-sourced data, we got 5,869 effective samples in total.

3.2 Prompt Crafting

To support our multi-stage investigation, we crafted two different prompt sets tailored to different research objectives. The **first set A** should provides a diverse array of real-world scenarios to evaluate the general readability of LLM-generated code and identify specific defect patterns (**RQ1** and **RQ2**), while the **second set B** should facilitate a controlled experimental environment for **RQ3**. This strategic separation allows us to maintain ecological validity for problem identification while ensuring the empirical rigor required for variable-controlled analysis of prompt dimensions.

3.2.1 Set A: Real-World Scenario Prompts (RQ1 & RQ2). This set establishes a baseline for general readability and defect identification. Specifically, we used the 5,869 samples previously collected from real software engineering practice as the foundational data, which includes 2,869 natural language problem descriptions from LeetCode and 3,000 function-level docstrings from WoC. To ensure consistency, two authors manually transformed the technical docstrings into coherent natural language instructions. This process involved independent transformation followed by a rigorous

¹To maintain a manageable research scope, in this study, we focused specifically on Python code readability.

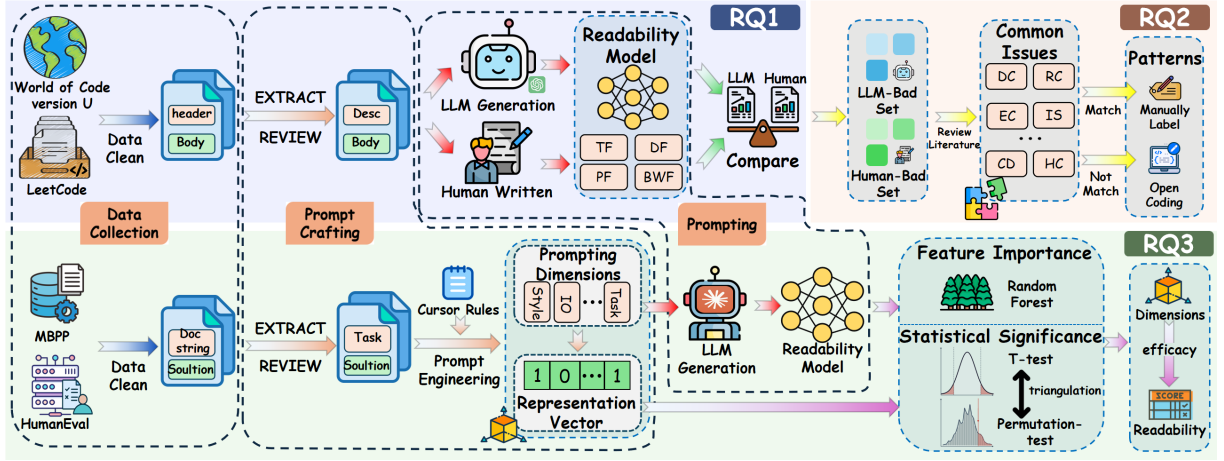


Figure 1: Overview of the methodology

cross-verification phase. Any discrepancies were resolved through iterative discussion to a consensus on all final expressions. We finally got a comprehensive set of 5,869 prompts reflecting diverse real-world development tasks.

3.2.2 Set B: Controlled Experimental Prompts (RQ3). Analyzing how specific prompt dimensions affect readability requires a rigorous control of variables. The complexity of real software practice in the first set makes it difficult to decouple specific descriptive elements into isolated variables. Thus, we adopted a from-scratch construction strategy, prioritizing simplicity and modularity to ensure experimental control.

We sourced problems from MBPP [8] and HumanEval [17] as they are established benchmarks with concise descriptions that are easily decoupled into base components. To maintain a balanced sample size, we combined 164 samples from HumanEval with a randomly sampled subset of 164 problems from MBPP. Following a similar refinement pipeline, since MBPP has provided direct natural language descriptions, we converted HumanEval docstrings into basic task descriptions, decoupling core logic from additional expressions to facilitate a controlled experiment. This process was followed by author cross-validation to ensure rigor and accuracy.

Referring to prior work [27, 93, 101], we limited the dimensions of prompt crafting to 8 types, as shown in Table 1. *Minimum task description* served as the baseline, consisting of the most basic problem description without any additional information. *Function signature*, *IO contract*, *few-shot instruction*, and *task category* were extracted from the code implementation and test cases, while *style description*, *persona setting*, and *constraint* were derived from best practices in cursor rules [7]. Starting with each problem’s baseline prompt, we constructed prompts that only added one dimension, omitted only one dimension, or included all dimensions and recorded dimensions that current prompt contained with a seven-element vector:

$$V_{\text{prompt}} = (\mathbb{I}\{d\})_{d \in \mathcal{D}},$$

where $\mathcal{D} = \{\text{style, signature, IO, few-shot, category, persona, constraint}\}$, and $\mathbb{I}\{d\} = 1$ if dimension d is included in the prompt, otherwise 0.

Finally, each baseline corresponded to 16 different prompts and vectors representing their crafting dimensions, ultimately yielding 5,248 operational (*prompt, vector*) pairs in set B.

3.3 Prompting and Data Processing

For the prompt set A (RQ1 & RQ2), we input the 5,869 prompts into a set of prevalent LLMs, including GPT-4o [2], Grok-3 [98], Claude-3.7 [5], DeepSeek-v3 [29], and Llama 3.1 [86], to obtain their responses. We extracted code body from the markdown-formatted text returned by LLMs and cleaned non-function parts, such as execution examples. We then applied the readability model to LLM-generated code and the corresponding human-written baseline. After that, we compared the readability between code authored by LLMs and humans, as well as between code generated by different LLMs using the same prompt.

Regarding the prompt set B (RQ3), to conserve computing resources, based on the readability assessment results, we selected the LLM that demonstrated the best code readability performance for further experimentation, which turned out to be Claude-3.7. We then inputted the 5,248 prompts to obtain the returned markdown-formatted text, and used similar methods to extract Python code and remove extraneous information. We then evaluated the readability of the generated code using our readability model.

3.4 Thematic Analysis

To address RQ2, we conducted a comparative and thematic analysis [84, 96] of LLM-generated code and human-written code. We first identified the most readable model outputs and then randomly sampled 500 pairs from both the WoC-sourced and LeetCode-sourced data for subsequent analysis. Based on readability scores, samples were categorized into those where LLMs outperformed humans (405 WoC, 328 LeetCode) and those where humans scored higher (95 WoC, 172 LeetCode). Subsequently, two authors independently read the code pairs and manually annotated them.

Specifically, each annotator first read both LLM and human-authored code in their entirety and identified which of the four basic

Table 1: Dimensions of Prompt Crafting

Dimension	Explanation
<i>Minimum task description</i>	The core instruction or objective of the prompt as brief and clear as possible.
<i>Style description</i>	Instructions specifying the tone, format and overall aesthetic of desired output.
<i>Function signature</i>	A specific instruction that forces the model’s output to conform to a structure, typically including the name, parameters, and type hint.
<i>IO Contract</i>	A precise description of the expected data structure, format, and content for both the LLM input and output.
<i>Few-shot instruction</i>	Providing the model with several examples of the input/output pairs for the specific task.
<i>Task category</i>	Explicitly naming the general type of problem or task the model is being asked to perform.
<i>Persona setting</i>	Assigning the model a specific identity, role, or character to adopt during the task execution.
<i>Constraint</i>	Restrictive guidelines specifying what the model must exclude or minimize.

readability dimensions exhibited poorer readability in the lower-scoring code, which include Textual Features (TF) [78], Posnett’s Features (PF) [72], Buse and Weimer’s Features (BWF) [15], and Dorn’s Features (DF) [24]. We detail these dimensions in Section 4. Subsequently, based on prior literature on code readability [43, 63, 66, 71, 78], we synthesized 9 common readability issue patterns, as shown in Table 2. Annotators identified readability defects in less readable snippets relative to their counterparts. Any issues not captured by the nine predefined patterns were addressed through open coding.

Independent annotations yielded Cohen’s Kappa values of 0.87 for dimensionality assessment and 0.81 for issue pattern identification, reflecting substantial inter-rater agreement. All discrepancies were resolved through discussions to reach consensus. For the open-coded segments, we synthesized individually derived codes into novel readability issue patterns distinct from the predefined ones. To ensure analytical rigor, we employed triangulation [34] across different data sources (WoC and LeetCode) and conducted negative case analysis [32] to refine our understanding of nuanced readability differences. This iterative process continued until theoretical saturation was reached, with no further novel insights emerging from the data.

3.5 Statistical Analysis

To address RQ3, we conducted a comprehensive statistical analysis on the state vectors of 5,248 prompts in set B and their corresponding readability scores of LLM-generated code. First, we performed a random forest regression analysis [12] across all samples, with $n_estimator = 100$, $max_depth = 15$, $min_samples_split = 5$. Random Forest’s non-linear modeling capability is well-suited for our analysis, and it provides feature importance rankings [38] that identify which prompt dimensions exert relatively greater influence on LLM-generated code readability scores.

Furthermore, we evaluated the statistical significance of each dimension’s impact through methodological triangulation, combining t-tests [61] and permutation importance testing [4]. While

Table 2: Common Readability Issue Patterns

Issue Pattern	Explanation
<i>Deficient Comment (DC)</i>	Critical or complex code lacks necessary explanations, purpose descriptions, or non-obvious details.
<i>Redundant Comment (RC)</i>	Comments merely duplicate information that is already clear from the code itself, offering no added value.
<i>Inconsistent Style (IS)</i>	The code fails to follow an established, uniform standard in naming conventions, indentation, formatting, or idiomatic usage.
<i>Excessive Complexity (EC)</i>	The logic, control flow, or structure of the code is overly convoluted, deeply nested, or uses unnecessarily complicated mechanisms.
<i>Poor Structure (PS)</i>	Code snippets have poor modularity, disorganized logic, or an unclear structural hierarchy.
<i>Poor Naming (PN)</i>	Names for variables, functions, classes, or other identifiers are vague, misleading, or fail to accurately reflect their true purpose or scope.
<i>Magic Values (MV)</i>	The code directly uses raw constant values (like numbers or strings) that lack explanatory context.
<i>Code Duplication (CD)</i>	Highly similar code fragments appear multiple times in different parts of the program, increasing maintenance cost and potential for errors.
<i>High Coupling (HC)</i>	Different functional parts are excessively dependent on one another, where a change in one component requires cascading changes in others.

t-tests identify significant readability score differences between baseline and incremental prompts ($p < 0.1$), permutation testing quantifies each dimension’s predictive contribution. Specifically, we utilized a random forest model as prediction kernel, and then shuffled the binary labels of the tested dimension across 10,000 iterations, finally considering a dimension relevant if its predictive performance significantly exceeded random chance ($p < 0.1$).

By synthesizing these two statistical perspectives, we determined whether each dimension significantly correlates with LLM-generated code readability. To ensure reliability, we also conducted an ablation study, which compared full-dimension prompts against those lacking a single dimension, to corroborate findings with inverse evidence.

4 Readability Model

To evaluate code readability, we combined established readability metrics from prior work into a unified representation that comprehensively quantifies readability and helps identify its issue patterns. Specifically, we constructed an integrated metric encompassing TF [78], PF [72], BWF [15], and DF [24]. This selection is based on extensive code readability literature [15, 24, 47, 72, 78, 79], representing widely adopted indicators of code comprehension.

Notably, these four families capture complementary perspectives on readability: TF focuses on textual and lexical characteristics, PF reflects information-theoretic and distributional properties of programs, BWF models formatting and whitespace cues, and DF captures visual presentation features. By synthesizing these diverse dimensions, our integrated metric provides a broad and complementary coverage of readability-related properties, reducing the bias of relying on any single metric family [78, 79].

Due to space constraints, the exhaustive technical details of our measurement framework, including mathematical formulations, metric selection criteria, and comprehensive model evaluation, are provided in appendix A for reference and reproducibility. In this section, we focus on the qualitative rationale of the selected metrics and highlight the pivotal aspects of the model’s implementation and evaluation.

4.1 Metric

We summarize all the metrics in our comprehensive readability model in Table A.1, which are grouped into TF, BWF, PF and DF. Checkmarks denote the computed statistics.

4.1.1 TF: Semantic and Lexical Perspective. To capture the impact of semantic and lexical features on code readability, we followed Scalabrino et al. [78, 79] to adopt TF features. This family is motivated by the observation that identifiers and comments are primary carriers of developer intent [46, 48]. Accordingly, TF captures whether the wording used in code is natural, semantically precise, and internally consistent with the documented behavior of the snippet. Representative signals in this family describe naming quality, comment–code consistency, lexical ambiguity, and conceptual cohesion. Collectively, these features reflect how easily a reader can correlate program text with the underlying problem domain and recover the intended meaning of the code implementation.

4.1.2 BWF: Structural Formatting Perspective. We employed BWF features following Buse and Weimer [15] to capture the formatting and layout characterization. This family focuses on low-level structural regularities that can be perceived directly from the arrangement of code on the screen without deep semantic analysis. In our setting, BWF summarizes three complementary aspects of structure: the horizontal density of individual lines, the syntactic distribution within lines, and the vertical segmentation induced by blank lines and comments. Collectively, these features reflect whether a snippet is visually crowded, unevenly distributed, or clearly partitioned into manageable units, all of which affect scanning effort and local comprehension [14, 15, 97].

4.1.3 PF: Information-Theoretic Perspective. To assess readability from an information-theoretic perspective, we incorporated PF features based on Posnett’s entropy-oriented formulation [72]. The underlying rationale is that code becomes harder to process when its information content is high or its token distribution is irregular. This family therefore captures the compactness, diversity, and unpredictability of the textual signal produced by the code. Representative features in this group quantify the overall information volume of a snippet and the degree of lexical disorder between lines. From a cognitive perspective, these measures approximate the amount of information that a reader must organize and retain when reading code [31, 72, 81].

4.1.4 DF: Visual and Geometric Perspective. To complement the aforementioned non-visual dimensions, we adopted DF features introduced by Dorn [24], which model code as a visual object. This family captures readability from a geometric and perceptual perspective by describing the shape, alignment, and visual distribution

of code on the page. In particular, DF reflects whether a snippet exhibits a regular spatial layout, consistent alignment of syntactic elements, and visually meaningful decoration patterns. Such properties influence how efficiently readers can detect visual patterns before fully parsing the text, as prior studies have suggested that spatial regularity and alignment facilitate code navigation [16, 19, 20, 80].

4.2 Implementation

To realize the metric families defined above, we implemented a unified pipeline that extracts measurable features from the abstract readability dimensions. In this stage, we focused on transforming raw code snippets into standardized readability representations, preparing for subsequent feature selection and evaluation. In particular, our workflow preserves layout-sensitive formatting to maintain the structural and visual indicators of readability, which are then parsed by language-aware extraction routines to derive essential signals for our readability metrics.

Building on these extracted signals, we recombined established metrics from prior work into a unified representation [62, 77, 78] to instantiate the four complementary families: TF, BWF, PF, and DF. All metrics are represented as snippet-level scalar features and concatenated in a fixed order to form a unified feature vector $\mathbf{x} = [\text{TF}, \text{BWF}, \text{PF}, \text{DF}]$. In our implementation, TF, BWF, PF, and DF yield 16, 26, 4, and 15 features, respectively, resulting in a raw 61-dimensional representation in total. This unified 61-dimensional representation serves as the input space for our downstream readability prediction and family-wise analysis.

To keep the main text focused on the overall workflow and its role in constructing the feature space, we defer implementation details to the appendix provided within the replication package. These details include preprocessing rules, language-specific extraction patterns, normalization procedures, and feature aggregation mechanisms.

4.3 Evaluation

To evaluate the efficacy of our code readability assessment framework, we conducted experiments on the Ddorn dataset [24], a widely used benchmark for code readability evaluation. In particular, the dataset consists of 360 code snippets across three programming languages: Java, Python, and CUDA. Each code snippet is labeled with binary readability scores based on human expert annotations.

We adopted a 10-fold stratified cross-validation protocol [45] to obtain a robust evaluation. Within each cross-validation fold, features are standardized, preventing features with larger scales from dominating the model. Given the high dimensionality of our initial 61-feature space, it is crucial to filter out noisy signals across different metric families. Therefore, we applied Sequential Forward Selection (SFS) [30, 94] to identify the discriminative subset of features. We utilized a Logistic Regression classifier with L2 regularization [10] as the base estimator for SFS. For each setting, SFS starts from the full candidate feature set and is performed within the training split of each fold. Accordingly, the Features column in Table 3 reports the optimal number of selected features per setting.

We reported two evaluation metrics, Accuracy and Area Under the ROC Curve (AUC) [26]. These metrics capture both classification accuracy and ranking quality. The evaluation covers the four

Table 3: Cross-validated performance of our readability assessment framework.

Feature Family	Features	Accuracy	AUC
TF	15	62.2%	65.3%
BWF	24	73.3%	79.4%
PF	3	66.4%	70.6%
DF	14	66.4%	76.2%
All-features	25	77.5%	83.8%

Table 4: Readability Scores of LLM-generated and Human-written Code

Model	Data Source	Avg (LLM)	Avg (Human)	Win Rate (%)
Claude-3.7	WoC	0.85	0.82	65.60
	LeetCode	2.15	1.45	81.00
	All	1.29	0.77	73.30
GPT-4o	WoC	0.45	0.09	67.54
	LeetCode	1.89	1.43	73.08
	All	1.00	0.61	69.67
Grok-3	WoC	0.24	0.09	57.60
	LeetCode	2.13	1.45	78.80
	All	1.18	0.77	68.20
DeepSeek-v3	WoC	0.44	0.09	64.26
	LeetCode	1.84	1.45	65.00
	All	1.14	0.77	64.63
Llama 3	WoC	0.33	0.14	57.25
	LeetCode	1.80	1.45	65.40
	All	1.13	0.86	61.71
All	WoC	0.38	0.10	62.63
	LeetCode	1.97	1.45	72.62
	All	1.15	0.76	67.52

* Avg refers to the average readability score of corresponding code snippets evaluated by the readability model; Win Rate refers to the proportion of samples where LLM-generated code strictly outperforms the human-written counterpart in readability.

individual feature families: TF, BWF, PF, and DF, as well as their concatenation, denoted as All-features.

Overall, the results in Table 3 indicate that our unified feature framework captures a richer set of readability signals than any single category. The All-features combination achieves peak performance (77.5% accuracy, 83.8% AUC), consistently outperforming individual feature families and previously proposed standalone perspectives [14, 24, 72, 79]. This highlights the complementary nature of our four dimensions and suggests that a multi-faceted representation is essential for capturing the full complexity of readability.

5 RQ1: Comparison of the Readability Scores

Following established methodology, we conducted a comparative readability analysis between LLM-generated and human-written code, with results summarized in Table 4 and Figure 2.

Table 4 shows that LLM-generated code consistently achieves higher average readability than human-written code across all models, outperforming humans in over 50% of cases. Although WoC-sourced samples exhibit lower absolute scores than LeetCode, which is likely due to the inherent complexity of real-world software scenarios, it does not undermine the validity of our relative comparison, as LLM and human outputs are evaluated within identical task contexts.

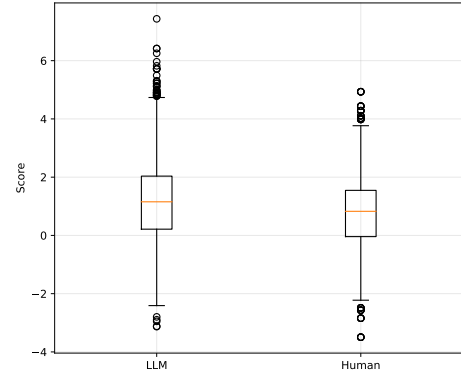
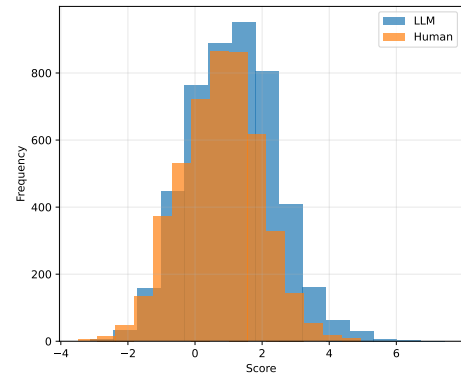
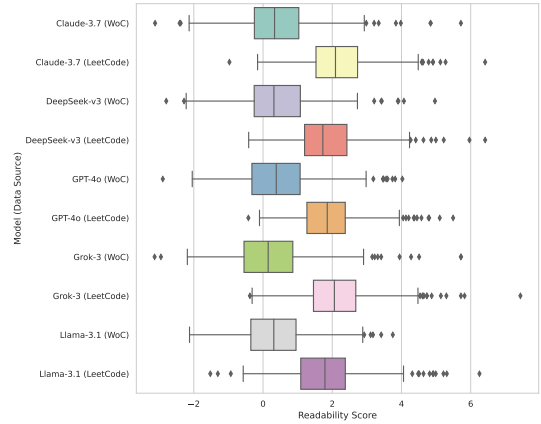
**(a) Boxplot of the readability scores****(b) Histogram of the readability scores****Figure 2: Comparison of the readability scores of LLM-generated code and Human-written code****Figure 3: Readability score distribution of code generated by different LLMs**

Figure 2 illustrates the readability score distributions, revealing that while LLMs achieve a marginally higher median, human-written code demonstrates superior consistency, as evidenced by a tighter interquartile range and fewer outliers (Figure 2a). This

phenomenon indicates that LLM-generated code has a less stable readability level, which may be attributed to the inherent next-token prediction mechanism [87] underlying LLM code generation, in contrast to programmers’ established programming habits [60]. Figure 2b demonstrates substantial distributional overlap of LLM-generated and human-written code, with marginally higher modes and peak scores in LLM outputs.

To establish a more rigorous comparison, we employed a Wilcoxon signed-rank test [95] on $N = 5,869$ matched pairs, as the paired design accounts for task-specific variance. The results reveal a statistically significant difference between two groups ($W/W_{max} = 0.270, p < 0.001$). The moderate effect size ($r = 0.398$) indicates a consistent but modest performance lead for LLM-generated code over human-written code when evaluated on identical tasks.

Thus, considering the large sample size and practical complexities, the readability of LLM-generated code is at least comparable to, if not slightly higher than, that of human-written code.

Additionally, the 5,869 collected prompt-code pairs provide a benchmark enabling a cross-model evaluation of code readability. As shown in Figure 3, the results confirm that simpler LeetCode tasks yield more readable code than complex WoC scenarios across all models. Notably, Claude-3.7 emerges as the top performer, achieving the highest average readability with the lowest variance. Since LLM capacity improves constantly with its development, it is reasonable to focus on Claude-3.7 for subsequent analyses to ensure both representativeness and computational efficiency.

Summary for RQ1:

Analysis of 5,869 matched code pairs across WoC and LeetCode reveals that LLM-generated code achieves statistically higher readability than human-written code ($p < 0.001$) with a moderate effect size ($r = 0.398$). It suggests that they are at least comparable in practice. However, a key distinction lies in stability: LLMs reach higher peak readability, whereas human-written code exhibits superior consistency. By establishing this practical parity alongside the stability distinction, our findings validate the non-functional potential of LLMs for systematic software development, while our dataset provides a robust benchmark for future comparative evaluations.

6 RQ2: Distinct Readability Issue Patterns

Table 5 illustrates readability issues between LLM-generated and human-written code across the four analytical dimensions detailed in Section 4. To briefly recap, **TF** (Semantic/Lexical) evaluates intent and naming; **BWF** (Structural/Formatting) assesses layout and density; **PF** (Information-Theoretic) measures logical complexity and entropy; and **DF** (Visual/Geometric) captures spatial alignment. Our results indicate that poorly-rated human code primarily exhibits issues in BWF and TF, with PF being the least common, while worse-performing LLM code predominantly struggles with TF and PF, leaving BWF as the minor concern.

These distinct patterns reflect the underlying differences in code generation mechanisms. Human programmers often prioritize functional correctness and development efficiency over stylistic rigor

Table 5: Distribution of Issues on Readability Dimensions

Category	TF	BWF	PF	DF
Human_bad	72	76	<u>21</u>	45
LLM_bad	60	<u>26</u>	53	30
Total	132	102	<u>74</u>	75

Table 6: Categories of Readability Issue Patterns

Category	DC	RC	IS	EC	PS	PN	MV	CD	HC
Human_bad	60	<u>3</u>	58	41	24	22	7	6	<u>3</u>
LLM_bad	10	24	15	48	16	21	8	<u>2</u>	<u>0</u>
Total	70	27	73	89	40	43	15	<u>8</u>	<u>3</u>

and considerate textual expression, leading to BWF and TF issues [49, 76]. However, their experience-driven reliance on validated concise logic typically results in lower information entropy and predictable structures, thereby minimizing PF issues [60]. In contrast, having been trained on substantial amounts of standardized code [2, 87], LLMs tend to generate style-standard code with less BWF issues, but their lack of experiential best practice often leads to correct but over-engineered solutions, manifesting as PF logic bloat [52, 103]. Furthermore, TF emerged as the most frequent issue dimension in LLM-generated code, which could be attributed to the lack of emphasis on semantic appropriateness in training data and model’s tendency to hallucinate text that is irrelevant to the actual code logic [40, 44].

Table 6 presents all readability issue patterns in the groups with worse readability. In human-written code, Deficient Comments (DC) and Inconsistent Style (IS) are the most common problems, while Redundant Comments (RC) and High Coupling (HC) are rare. This pattern reflects the tendency of programmers to prioritize functional correctness and speed over non-essential tasks like documentation and styling [49]. Regarding comments as secondary to execution, developers often omit necessary explanations or simplify them too much, which leads to high DC but very few RC issues [76]. Similarly, high IS suggests that developers may neglect consistent formatting while focusing on solving the problem at hand.

The most prominent issue in LLM-generated code is Excessive Complexity (EC), while Code Duplication (CD) and High Coupling (HC) are infrequent. Unlike humans, LLMs do not inherently aim for the simplest solution. Instead, they often generate over-complicated code that is technically correct but difficult to follow, significantly leading to EC issues [52, 103]. Furthermore, while humans tend to skip comments, LLMs often produce Redundant Comments (RC) by following “add comments” instructions too literally, which instead reduce clarity [44]. The low incidence of CD suggests that LLMs are good at modularizing code and reusing logic without repetition. Finally, both human and LLM code show very low levels of HC. This is likely because our study focuses on function-level snippets, which naturally avoid HC phenomenon. Identifying HC typically requires analyzing file-level code, which is beyond the scope of current readability models. We further discuss it in Section 8.

Beyond predefined patterns, our open coding revealed three novel issues specific to LLM-generated code: **Unknown API**, **Redundant Variables**, and **Overblanking**. Unknown API occurs when LLMs introduce external library functions without adequate explanation. Whether these APIs are hallucinations or real, their inclusion often makes the code difficult to follow, likely because models rely on vast training knowledge without knowing whether the referenced libraries are accessible or familiar to the end user [75, 89]. Redundant Variables refer to cases where LLMs define multiple variable names for the same functional purpose, leading to confusion and unnecessary repetition. It likely stems from LLMs’ token-by-token generation, where context tracking of previously defined variables may be insufficient [52]. Overblanking involves excessive but meaningless blank lines that unnecessarily lengthen the program. Though related to code style, its high prevalence in LLM-generated code warrants a separate category, suggesting that LLMs often mimic human formatting patterns without understanding the actual structural or semantic purpose of the spacing.

Summary for RQ2:

LLM-generated and human-written code exhibit distinct readability issue patterns. Poorly-rated human code primarily suffers from structural (BWF) and semantic (TF) issues, specifically DC and IS, reflecting a prioritization of functional correctness over stylistic rigor, while LLMs predominantly struggle with semantic (TF) and information-theoretic (PF) dimensions, characterized by EC and RC. Notably, we identified three novel LLM-specific patterns: Unknown API, Redundant Variables, and Overblanking, which stem from LLM generation mechanisms rather than human coding habits.

7 RQ3: Prompt Efficacy on Code Readability

Figure 4 shows the feature importance F-scores of random forest regression. *Function signature* demonstrates the highest importance among all evaluated prompt dimensions, followed by *constraints* and *style description*, while *task category* and *IO contract* play minimal roles. This hierarchy suggests that *function signatures* establish the stylistic foundation for function-level snippets [22], further influencing readability, while *style descriptions* and *constraints* provide

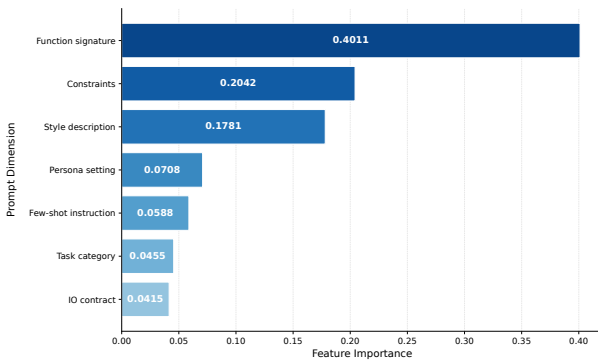


Figure 4: Feature importance of the Random Forest regression result

Table 7: P-value of Statistic Analysis of Prompt Dimensions and Code Readability

Dimension	Incremental		Ablation	
	T-test	Permutation	T-test	Permutation
<i>Style description</i>	0.001	0.001	0.795	0.850
<i>Function signature</i>	0.239	0.240	0.003	0.003
<i>IO Contract</i>	0.353	0.356	0.855	0.887
<i>Few-shot</i>	0.257	0.291	0.643	0.678
<i>Task category</i>	0.282	0.322	0.673	0.684
<i>Persona setting</i>	0.287	0.282	0.281	0.287
<i>Constraint</i>	0.001	0.000	0.807	0.960

* Bold values indicate statistical significance ($p < 0.05$).

complementary explicit guidance, offering “do’s” and “don’ts” for textual clarity that aligns with intuitive expectations. Conversely, *task category* and *IO contract* appear to drive functional logic and problem-solving rather than influence readability. Notably, the low R^2 (< 0.3) of our random forest model indicates that prompt engineering has a limited impact on readability scores. As described in Section 3, to substantiate this observation, we conducted statistical significance tests and ablation analysis to isolate the precise effects of each dimension, with results shown in Table 7.

Our incremental analysis, which compares baseline prompts with single-dimension additions, shows that *style descriptions* and *constraints* significantly correlate with improved code readability. This suggests that when starting from a minimal task description, explicitly declaring readability requirements, whether through positive directives or negative restrictions, maximally improves code readability. On the other hand, our ablation analysis reveals that when using comprehensive prompts, only *function signature* maintains a significant correlation with readability. This indicates that in well-designed prompts, *function signature* is crucial for output quality. We hypothesize that signatures help define the “contextual domain” of the generation process, guiding the LLM to adhere to better readability standards when other instructions are relatively complete [22]. These results strongly reinforce our earlier findings through random forest regression.

Synthesizing these analyses, we found that *function signatures*, *constraints*, and *style descriptions* are the three most influential prompt dimensions for code readability. Specifically, *constraints* and *style descriptions* help simple prompts reach their maximum potential, while *function signatures* ensure more readable outputs in well-designed prompts. It is important to note that from an absolute perspective, the overall impact of prompt engineering on readability is relatively modest compared to other dominant factors like problem complexity and model capability [3, 52]. However, for users seeking to improve code readability at minimal cost, optimizing these three specific prompt dimensions provides a practical and valuable strategy.

Summary for RQ3:

Among the seven evaluated dimensions, *function signatures*, *style descriptions*, and *constraints* emerged as the most influential factors. Specifically, *style descriptions* and *constraints* enhance basic prompts through explicit “do’s” and “don’ts,” while *function signatures* serve as a vital contextual anchor in more complex prompts. However, prompt engineering alone remains a relatively weak strategy for improving code readability, highlighting the need for more specialized solutions.

8 Discussion

8.1 Implications

Our findings offer practical insights for advancing automated software engineering tasks, specifically in tool development, model evaluation, and management of AI-driven software evolution.

Multi-dimensional Model Evaluation and Optimization. Current automated evaluation benchmarks for code-generating LLMs focus almost exclusively on functional correctness and task completion rate via unit tests [55, 90]. However, our finding that LLMs achieve numerical readability parity with humans while introducing distinct readability issues highlights significant potential for LLMs to enter systematic software development, but a novel critical correctness-readability gap. Models that generate functionally correct but cryptic code ultimately impose greater cognitive burdens on developers in code maintenance [39]. Thus, we advocate for a multi-dimensional automated evaluation approach. Future benchmarks should integrate non-functional metrics such as readability into their optimization objectives. For model developers, incorporating these readability metrics into the RLHF loop could help automate the generation of code that is not only correct but also architecturally elegant and easier to read.

Automated Code Review. Our identification of distinct readability issues in LLM-generated code suggest that traditional static analysis tools [68], which are primarily designed to catch human-typical “bad smells,” may be insufficient for the LLM era. Our results show that, besides LLM-specific readability issues (i.e., Unknown API usage, Redundant Variables, and Overblanking), there is a fundamental divergence: while human-typical readability issues often involve omission (e.g., *Deficient Comments*), LLM-generated code suffers from commission (e.g., *Excessive Complexity*). This inspires the development of AI-specific automated reviewers, which should be designed to detect logical bloat, unnecessary variable redundancies, and hallucinated or obscure API usage. Integrating these specialized rules into CI/CD pipelines can help automate the quality gate, ensuring that the productivity gains of AI-assisted coding do not come at the cost of future maintainability [9].

LLM-Generated Code Detection. As LLM-generated code increasingly populates open-source ecosystems, the ability to automate code attribution becomes crucial for security and licensing [40]. Our results suggest that high-level readability patterns provide more interpretable and robust signals besides low-level statistical markers (e.g., token distribution) used in traditional detection methods. ASE researchers can leverage these stylistic and structural fingerprints to build explainable automated detectors. Such tools would not only identify the provenance of a code snippet

but also provide a rationale based on readability characteristics, which is important for transparency in software supply chains.

Technical Debt in AI-Assisted Development. The emergence of novel readability issue patterns in LLM-generated code indicates a new form of AI-driven technical debt [85]. Traditional approaches to identifying and measuring code quality issues may inadequately capture LLM-specific problems, potentially leading to underestimation of maintenance burdens in AI-assisted development projects. Automated software maintenance tools should be updated to account for these specific AI-generated patterns. Specifically, automated refactoring agents could be trained to optimize LLM output in a targeted manner, such as collapsing redundant logic, to align LLM-generated code with project-specific readability standards before it enters the codebase.

Towards Improving Code Readability. Our analysis of prompt design effectiveness reveals that while overall impact remains limited, specific dimensions like *function signature*, *style description*, and *constraints* are relatively most efficient for readability improvement. As a low-cost strategy, practitioners should focus their prompt optimization efforts on these high-impact dimensions to maximize efficiency. Meanwhile, the limited effectiveness of the identified prompt design dimensions in improving readability shed light on the need for alternative approaches, such as post-processing techniques [70], fine-tuning strategies specifically targeting readability objectives [91], or hybrid human-AI collaborative coding workflows [33] to better improve the readability level of LLM-generated code, finally fostering a healthier software ecosystem.

8.2 Threats to Validity

8.2.1 Internal Validity. A primary threat to internal validity concerns the challenge of capturing the subjective and multifaceted nature of code readability. Although our model integrates dimensions from validated prior work, it may not be exhaustive, as individual developer preferences vary and cannot be fully measured by a standardized set of metrics. Furthermore, inherent complexity differences between our data sources, where WoC represents production-grade code and LeetCode focuses on constrained algorithmic challenges, inevitably influence code structure and require caution when interpreting cross-source comparisons [77]. Finally, our focus on function-level snippets limits the detection of high coupling, which typically manifests at the file or project level. Therefore, its low prevalence in our findings reflects our chosen granularity rather than an absence of the defect in real-world scenarios.

8.2.2 External Validity. The generalizability of our findings is limited by the focus on Python and a specific subset of LLMs, as results may vary for languages with different syntactic conventions [82] or for models specifically fine-tuned for code generation. Additionally, although our sample is substantial, it may not fully reflect the complexities of production environments or the iterative nature of human-AI collaboration. Real-world code generation often involves multi-round interactions and strict project-specific constraints that independent snippets cannot capture, highlighting the need for future research into collaborative, large-scale software engineering.

9 Conclusion

This study evaluates the readability of LLM-generated code against human baselines. While numerical metrics suggest readability parity, our analysis moves beyond readability scores to reveal fundamental qualitative divergences. Poorly-rated human code is primarily characterized by *Deficient Comments* and *Inconsistent Style*, whereas LLM-generated code predominantly exhibits *Excessive Complexity* and *Redundant Comments*. Additionally, we identified novel, AI-specific issue patterns, such as Unknown API usage, Redundant Variables, and Overblanking, which may correlate with the inherent mechanisms of generative models. Furthermore, while *function signatures*, *style descriptions*, and *constraints* are the most influential prompt dimensions to output readability, overall impact of prompt engineering remains modest. These findings validate the potential for systematic AI integration into software workflows from a non-functional perspective while highlighting its latent technical debt, encouraging future research into approaches to enhance the readability of LLM-generated code to ensure long-term maintainability.

References

- [1] [n. d.]. LeetCode. <https://leetcode.com/problemset/>.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Duaa Alawad, Manisha Panta, Minhaz Zibran, and Md Rakibul Islam. 2019. An empirical study of the relationships between code readability and software complexity. *arXiv preprint arXiv:1909.01760* (2019).
- [4] André Altmann, Laura Toloşi, Oliver Sander, and Thomas Lengauer. 2010. Permutation importance: a corrected feature importance measure. *Bioinformatics* 26, 10 (2010), 1340–1347.
- [5] Anthropic. [n. d.]. Claude. <https://www.anthropic.com/claude/>.
- [6] Anysphere. [n. d.]. Cursor. <https://www.cursor.com/>.
- [7] Ashley. 2024. What is .cursorrule and How to Use It Effectively. <https://medium.com/towards-agi/what-are-cursor-rules-and-how-to-use-them-ec558468d139>.
- [8] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [9] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 712–721.
- [10] Christopher M Bishop and Nasser M Nasrabadi. 2006. *Pattern recognition and machine learning*. Vol. 4. Springer.
- [11] Dustin Boswell and Trevor Foucher. 2011. *The art of readable code*. " O'Reilly Media, Inc".
- [12] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32.
- [13] Kiran Busch, Alexander Rochlitz, Diana Sola, and Henrik Leopold. 2023. Just tell me: Prompt engineering in business process management. In *International Conference on Business Process Modeling, Development and Support*. Springer, 3–11.
- [14] Raymond PL Buse and Westley R Weimer. 2008. A metric for software readability. In *Proceedings of the 2008 international symposium on Software testing and analysis*. 121–130.
- [15] Raymond PL Buse and Westley R Weimer. 2009. Learning a metric for code readability. *IEEE Transactions on software engineering* 36, 4 (2009), 546–558.
- [16] Teresa Busjahn, Roman Bednarič, Andrew Begel, Martha Crosby, James H Pateron, Carsten Schulte, Bonita Sharif, and Sascha Tamm. 2015. Eye movements in code reading: Relaxing the linear order. In *2015 IEEE 23rd international conference on program comprehension*. IEEE, 255–265.
- [17] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [18] Tristan Coignon, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 79–89.
- [19] Martha E Crosby and Jan Stelovsky. 2002. How do we read algorithms? A case study. *Computer* 23, 1 (2002), 25–35.
- [20] Pablo Roberto Fernandes de Oliveira, Rohit Gheyi, José Aldo Silva da Costa, and Márcio Ribeiro. 2024. Assessing Python Style Guides: An Eye-Tracking Study with Novice Developers. In *Simpósio Brasileiro de Engenharia de Software (SBES)*. SBC, 136–146.
- [21] George Digkas, Alexander Chatzigeorgiou, Apostolos Ampatzoglou, and Paris Avgeriou. 2020. Can clean new code reduce technical debt density? *IEEE Transactions on Software Engineering* 48, 5 (2020), 1705–1721.
- [22] Xi Ding, Rui Peng, Xiangping Chen, Yuan Huang, Jing Bian, and Zibin Zheng. 2024. Do code summarization models process too much information? function signature may be all that is needed. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–35.
- [23] John Dooley and John Zukowski. 2011. *Software development and professional practice*. Springer.
- [24] Jonathan Dorn. 2012. A general software readability model. *MCS Thesis available at (web.eecs.umich.edu/~weimerw/students/dorn-mcs-paper.pdf)* 5 (2012), 11–14.
- [25] James L Elshoff and Michael Marcotty. 1982. Improving computer program readability to aid modification. *Commun. ACM* 25, 8 (1982), 512–521.
- [26] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern recognition letters* 27, 8 (2006), 861–874.
- [27] Louie Giray. 2023. Prompt engineering with ChatGPT: a guide for academic writers. *Annals of biomedical engineering* 51, 12 (2023), 2629–2633.
- [28] Github. [n. d.]. Copilot. <https://github.com/features/copilot>.
- [29] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Quanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [30] Isabelle Guyon and André Elisseeff. 2003. An introduction to variable and feature selection. *Journal of machine learning research* 3, Mar (2003), 1157–1182.
- [31] Maurice H Halstead. 1977. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc.
- [32] Ardis Hanson. 2017. Negative case analysis. *The international encyclopedia of communication research methods* (2017), 1–2.
- [33] Mohammad Hassany, Jiaze Ke, Peter Brusilovsky, Arun Balajee Lekshmi Narayanan, and Kamil Akhuseynoglu. 2024. Authoring Worked Examples for JAVA Programming with Human AI Collaboration. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 101–103.
- [34] Roberta Heale and Dorothy Forbes. 2013. Understanding triangulation in research. *Evidence-based nursing* 16, 4 (2013), 98–98.
- [35] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [36] Chao Hu, Yitian Chai, Hao Zhou, Fandong Meng, Jie Zhou, and Xiaodong Gu. 2024. How Effectively Do Code Language Models Understand Poor-Readability Code?. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 795–806.
- [37] Yuan Huang, Nan Jia, Junhui Shu, Xinyu Hu, Xiangping Chen, and Qiang Zhou. 2020. Does your code need comment? *Software: Practice and Experience* 50, 3 (2020), 227–245.
- [38] Reza Iranzad and Xiao Liu. 2025. A review of random forest-based feature selection methods for data science education and applications. *International Journal of Data Science and Analytics* 20, 2 (2025), 197–211.
- [39] Ciera Jaspán and Collin Green. 2023. Defining, measuring, and managing technical debt. *IEEE Software* 40, 03 (2023), 15–19.
- [40] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.
- [41] Can Jin, Hongwu Peng, Shiyu Zhao, Zhengting Wang, Wujiang Xu, Ligong Han, Jiahui Zhao, Kai Zhong, Sanguthevar Rajasekaran, and Dimitris N Metaxas. 2025. Apeer: Automatic prompt engineering enhances large language model reranking. In *Companion Proceedings of the ACM on Web Conference 2025*. 2494–2502.
- [42] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024).
- [43] John Johnson, Sergio Lubo, Nishitha Yedla, Jairo Aponte, and Bonita Sharif. 2019. An empirical study assessing source code readability in comprehension. In *2019 IEEE International conference on software maintenance and evolution (ICSME)*. IEEE, 513–523.
- [44] Sungmin Kang, Louis Milliken, and Shin Yoo. 2024. Identifying inaccurate descriptions in llm-generated code comments via test execution. *arXiv preprint arXiv:2406.14836* (2024).
- [45] Ron Kohavi et al. 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Ijcai*, Vol. 14. Montreal, Canada, 1137–1145.
- [46] Dawn Lawrie, Henry Feild, and David Binkley. 2006. Syntactic identifier conciseness and consistency. In *2006 Sixth IEEE International Workshop on Source Code Analysis and Manipulation*. IEEE, 139–148.

- [47] Dawn Lawrie, Christopher Morrell, Henry Feild, and David Binkley. 2006. What's in a Name? A Study of Identifiers. In *14th IEEE international conference on program comprehension (ICPC '06)*. IEEE, 3–12.
- [48] Dawn Lawrie, Christopher Morrell, Henry Feild, and David Binkley. 2007. Effective identifier names for comprehension and memory. *Innovations in Systems and Software Engineering* 3, 4 (2007), 303–318.
- [49] Valentina Lenarduzzi, Terese Besker, Davide Taibi, Antonio Martini, and Francesca Arcelli Fontana. 2021. A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools. *Journal of Systems and Software* 171 (2021), 110827.
- [50] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [51] Weixin Liang, Mert Yuksekgonul, Yining Mao, Eric Wu, and James Zou. 2023. GPT detectors are biased against non-native English writers. *Patterns* 4, 7 (2023).
- [52] Sherlock A Licorish, Ansh Bajpai, Chetan Arora, Fanyu Wang, and Kla Tantithamthavorn. 2025. Comparing Human and LLM Generated Code: The Jury is Still Out! *arXiv preprint arXiv:2501.16857* (2025).
- [53] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [54] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys* 55, 9 (2023), 1–35.
- [55] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).
- [56] Michael R Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamon Thongtanunam. 2024. Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology* (2024).
- [57] Yuxing Ma, Chris Bogart, Sadika Amreen, Russell Zaretzki, and Audris Mockus. 2019. World of code: an infrastructure for mining the universe of open source VCS data. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 143–154.
- [58] Yuxing Ma, Tapajit Dey, Chris Bogart, Sadika Amreen, Marat Valiev, Adam Tutko, David Kennard, Russell Zaretzki, and Audris Mockus. 2021. World of code: enabling a research workflow for mining and analyzing the universe of open source VCS data. *Empirical Software Engineering* 26 (2021), 1–42.
- [59] Ggaliwango Marvin, Nakayiza Hellen, Daudi Jjingo, and Joyce Nakatumba-Nabende. 2023. Prompt engineering in large language models. In *International conference on data intelligence and cognitive informatics*. Springer, 387–402.
- [60] Orni Meerbaum-Salant, Michal Armoni, and Mordechai Ben-Ari. 2011. Habits of programming in scratch. In *Proceedings of the 16th annual joint conference on Innovation and technology in computer science education*. 168–172.
- [61] Prabhakar Mishra, Uttam Singh, Chandra M Pandey, Priyadarshini Mishra, and Gaurav Pandey. 2019. Application of student's t-test, analysis of variance, and covariance. *Annals of cardiac anaesthesia* 22, 4 (2019), 407–411.
- [62] Leon Moonen. 2001. Generating robust parsers using island grammars. In *Proceedings eighth working conference on reverse engineering*. IEEE, 13–22.
- [63] Delano Oliveira, Reyndre Santos, Benedito De Oliveira, Martin Monperrus, Fernando Castor, and Fernanda Madeiral. 2024. Understanding Code Understandability Improvements in Code Reviews. *IEEE Transactions on Software Engineering* (2024).
- [64] OpenAI. [n.d.]. ChatGPT release. <https://openai.com/index/chatgpt/>.
- [65] Andy Oram and Greg Wilson. 2007. *Beautiful code Leading programmers explain how they think*. O'Reilly Media, Inc.
- [66] Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, and Andrea De Lucia. 2014. Do they really smell bad? a study on developers' perception of bad code smells. In *2014 IEEE International conference on software maintenance and evolution*. IEEE, 101–110.
- [67] Dangfeng Pan, Zhensu Sun, Cenyuan Zhang, David Lo, and Xiaoning Du. 2025. The Hidden Cost of Readability: How Code Formatting Silently Consumes Your LLM Budget. *arXiv preprint arXiv:2508.13666* (2025).
- [68] Sebastiano Panichella, Venera Arnaoudova, Massimiliano Di Penta, and Giuliano Antoniol. 2015. Would static analysis tools help developers with code reviews?. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 161–170.
- [69] Ethan Perez, Douwe Kiela, and Kyunghyun Cho. 2021. True few-shot learning with language models. *Advances in neural information processing systems* 34 (2021), 11054–11070.
- [70] Felix Petersen, Debarghya Mukherjee, Yuekai Sun, and Mikhail Yurochkin. 2021. Post-processing for individual fairness. *Advances in Neural Information Processing Systems* 34 (2021), 25944–25955.
- [71] Valentina Piantadosi, Fabiana Fierro, Simone Scalabrino, Alexander Serebrenik, and Rocco Oliveto. 2020. How does code readability change during software evolution? *Empirical Software Engineering* 25 (2020), 5374–5412.
- [72] Daryl Posnett, Abram Hindle, and Premkumar Devanbu. 2011. A simpler model of software readability. In *Proceedings of the 8th working conference on mining software repositories*. 73–82.
- [73] Vaclav Rajlich and Prashant Gosavi. 2004. Incremental change in object-oriented programming. *IEEE software* 21, 4 (2004), 62–69.
- [74] David Ramel. 2023. GitHub Copilot AI Tech Upgraded, Already Generates 61% of Java Code. <https://visualstudiomagazine.com/articles/2023/02/15/copilot-upgrade.aspx>.
- [75] Vipula Rawte, Amit Sheth, and Amitava Das. 2023. A survey of hallucination in large foundation models. *arXiv preprint arXiv:2309.05922* (2023).
- [76] Rodrigo REBOUÇAS DE ALMEIDA, Christoph TREUDE, and Uirá KULESZA. 2023. What's behind tight deadlines? Business causes of technical debt.(2023). In *Proceedings of the 16th International Conference on Cooperative and Human Aspects of Software Engineering, Melbourne, Australia*. 14–15.
- [77] Simone Scalabrino, Gabriele Bavota, Christopher Vendome, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2017. Automatically assessing code understandability: How far are we?. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 417–427.
- [78] Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. 2018. A comprehensive model for code readability. *Journal of Software: Evolution and Process* 30, 6 (2018), e1958.
- [79] Simone Scalabrino, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2016. Improving code readability models with textual features. In *2016 IEEE 24th International Conference on Program Comprehension (ICPC)*. IEEE, 1–10.
- [80] Mor Shamy and Dror G Feitelson. 2023. Identifying lines and interpreting vertical jumps in eye tracking studies of reading text and code. *ACM Transactions on Applied Perception* 20, 2 (2023), 1–20.
- [81] Claude E Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal* 27, 3 (1948), 379–423.
- [82] Andreas Steffik and Susanna Siebert. 2013. An empirical investigation into programming language syntax. *ACM Transactions on Computing Education (TOCE)* 13, 4 (2013), 1–40.
- [83] Wannita Takerungsaksiri, Chakkrit Tantithamthavorn, Micheal Fu, Jirat Paksukmit, Kun Chen, and Ming Wu. 2025. Code readability in the age of large language models: An industrial case study from atlassian. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 732–742.
- [84] Gareth Terry, Nikki Hayfield, Victoria Clarke, Virginia Braun, et al. 2017. Thematic analysis. *The SAGE handbook of qualitative research in psychology* 2, 17–37 (2017), 25.
- [85] Edith Tom, Aybüke Aurum, and Richard Vidgen. 2013. An exploration of technical debt. *Journal of Systems and Software* 86, 6 (2013), 1498–1516.
- [86] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [87] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [88] Antonio Vitale, Emanuela Guglielmi, Rocco Oliveto, and Simone Scalabrino. 2025. Personalized Code Readability Assessment: Are We There Yet? *arXiv preprint arXiv:2503.07870* (2025).
- [89] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. 2024. Llm meet library evolution: Evaluating deprecated api usage in llm-based code completion. *arXiv preprint arXiv:2406.09834* (2024).
- [90] Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A Smith, Iz Beltagy, et al. 2023. How far can camels go? exploring the state of instruction tuning on open resources. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 74764–74786.
- [91] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652* (2021).
- [92] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [93] J White. 2023. A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv preprint arXiv:2302.11382* (2023).
- [94] A Wayne Whitney. 1971. A direct method of nonparametric measurement selection. *IEEE transactions on computers* 100, 9 (1971), 1100–1103.
- [95] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83.

- [96] Michael Williams and Tami Moser. 2019. The art of coding and thematic exploration in qualitative research. *International management review* 15, 1 (2019), 45–55.
- [97] Scott N Woodfield, Hubert E Dunsmore, and Vincent Y Shen. 1981. The effect of modularization and comments on program comprehension. In *Proceedings of the 5th international conference on Software engineering*. 215–223.
- [98] xAI. [n. d.]. Grok. <https://grok.com/>.
- [99] Weiwei Xu, Kai Gao, Hao He, and Minghui Zhou. 2025. Licoeval: Evaluating llms on license compliance in code generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1665–1677.
- [100] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* (2024), 100211.
- [101] Xiang Zhang, Juntai Cao, Chenyu You, and Dujian Ding. 2025. Why prompt design matters and works: A complexity analysis of prompt search space in llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 32525–32555.
- [102] Li Zhong and Zilong Wang. 2024. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 21841–21849.
- [103] Yangtian Zi, Luisa Li, Arjun Guha, Carolyn Anderson, and Molly Q Feldman. 2025. “I Would Have Written My Code Differently”: Beginners Struggle to Understand LLM-Generated Code. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1479–1488.

A Readability Model

Table A.1: Construction of the Readability Model

DIMENSION	FEATURE	MIN	AVG	MAX	STD
TF	CIC (Comment-Identifier Consistency)		✓		
	CIC_syn (Synonym-expanded CIC)		✓		
	ITID (Identifier-Term in Dictionary)	✓	✓	✓	
	NMI (Narrow Meaning Identifiers)	✓	✓	✓	
	CR (Comment Readability)		✓		
	NM (Number of meanings)		✓	✓	
	TC (Text Coherence)	✓	✓	✓	
	NOC (Number of Concepts)		✓		
BWF	NOC_norm (Normalized NOC)		✓		
	Line length (characters)		✓	✓	
	Identifier length (characters)		✓	✓	
	N. of identifiers		✓	✓	
	Indentation (preceding whitespace)		✓	✓	
	N. of keywords		✓	✓	
	N. of numbers		✓	✓	
	N. of parentheses		✓		
	N. of brackets		✓		
	N. of periods		✓		
	N. of blank lines		✓		
	N. of comments		✓		
	N. of commas		✓		
	N. of spaces		✓		
	N. of assignments		✓		
	N. of branches (if)		✓		
	N. of loops (for, while)		✓		
	N. of arithmetic operators		✓		
	N. of comparison operators		✓		
	N. of occurrences of any character			✓	
	N. of occurrences of any identifier			✓	
PF	LOC (Lines of Code)		✓		
	Entropy		✓		✓
	Halstead Volume		✓		
DF	Visual keyword density		✓		
	Visual string density		✓		
	Visual comment density		✓		
	Visual complexity		✓		
	Spatial absolute area		✓		
	Spatial relative area		✓		
	Spatial regularity		✓		
	Spatial density		✓		
	Alignment operators		✓		
	Alignment brackets		✓		
	Alignment consistency		✓		
	Text English ratio		✓		
	Text comment ratio		✓		
	Text vocabulary size		✓		
	Text avg identifier length		✓		

A.1 Formal definitions of all metrics

This section outlines the implementation-aligned definitions of the metrics used in our readability model. Since several feature families are instantiated through lightweight heuristics rather than full parsing, the formulas below follow the actual computation rules of our implementation. In addition, detailed symbols used throughout the appendix are summarized in Table A.2.

Let a code snippet be denoted by

$$s = \{l_1, l_2, \dots, l_m\}, \quad (\text{A.1})$$

where l_i is the i -th line and m is the total number of lines. Let $T(s)$, $I(s)$, and $C(s)$ denote the lexical-token collection, identifier-token collection, and comment-token collection extracted from s , respectively. Here, $|I(s)|$ counts multiplicity. Let m_{ne} , m_{code} , and

Table A.2: Notation Summary

Symbol	Description
$s = \{l_1, \dots, l_m\}$	Code snippet with m lines; l_i is the i -th line
$m, m_{\text{ne}}, m_{\text{code}}, m_{\text{val}}$	Total, non-empty, code, and valid line counts
$T(s), I(s), C(s)$	Token collections with multiplicity for lexical tokens, identifiers, and comments
$T_c(s), T_i(s)$	Normalized term collections extracted from comments and identifiers
G_r, D	Identifier-term group for local unit r ; WordNet dictionary
$\mathcal{V}_{\text{char}}(s), \mathcal{V}_{\text{char}}(l_i), \mathcal{V}_{\text{id}}(s)$	Character vocabularies (snippet / line) and identifier vocabulary
$N_{\text{char}}, N_{\text{kw}}, N_{\text{str}}, N_{\text{com}}$	Numbers of total, keyword, string, and comment characters
W, S, Y	Numbers of words, sentences, and syllables in comment text
$b_i, C(s)$	Local textual block; cluster set from DBSCAN
$P_{\text{asg}}, P_{\text{br}}$	Column positions of assignment operators and brackets
$\mathbf{x} \in \mathbb{R}^{61}, \mathbf{x}'_j$	Raw feature vector and normalized feature value
$\mu_j, \sigma_j, \mathbf{w}, b$	Feature statistics (mean/std) and Logistic Regression parameters
$\sigma(\cdot)$	Logistic sigmoid function
$\text{avg}(\cdot), \text{std}(\cdot), \text{min}(\cdot), \text{max}(\cdot)$	Aggregation operators

m_{val} denote the numbers of non-empty lines, non-empty code lines, and valid extracted lines used by the corresponding feature module, respectively.

For a finite collection of local values $\mathcal{Z} = \{z_1, \dots, z_n\}$, we use the standard aggregation operators

$$\begin{aligned}
 \text{avg}(\mathcal{Z}) &= \frac{1}{n} \sum_{i=1}^n z_i, \\
 \text{std}(\mathcal{Z}) &= \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \text{avg}(\mathcal{Z}))^2}, \\
 \text{min}(\mathcal{Z}) &= \min_{1 \leq i \leq n} z_i, \\
 \text{max}(\mathcal{Z}) &= \max_{1 \leq i \leq n} z_i.
 \end{aligned} \quad (\text{A.2})$$

Unless otherwise specified, undefined quantities caused by missing evidence are assigned the default value 0 in the implementation.

A.1.1 TF. TF captures lexical quality, semantic precision, comment-code consistency, and conceptual coherence.

CIC and CIC_syn. Let $T_c(s)$ be the set of normalized terms extracted from all comments in s , and let $T_i(s)$ be the set of normalized identifier terms extracted from the entire snippet. Then

$$\text{CIC}(s) = \frac{|T_c(s) \cap T_i(s)|}{|T_c(s) \cup T_i(s)|}. \quad (\text{A.3})$$

For synonym-expanded matching, let $S_{\text{syn}}(A)$ denote the union of WordNet synonym terms associated with term set A . The implementation expands both sides:

$$\begin{aligned}\tilde{T}_c(s) &= T_c(s) \cup S_{\text{syn}}(T_c(s)), \\ \tilde{T}_i(s) &= T_i(s) \cup S_{\text{syn}}(T_i(s)), \\ \text{CIC}_{\text{syn}}(s) &= \frac{|\tilde{T}_c(s) \cap \tilde{T}_i(s)|}{|\tilde{T}_c(s) \cup \tilde{T}_i(s)|}.\end{aligned}\quad (\text{A.4})$$

ITID, NMI, and NM. Let $G_r \subseteq I(s)$ denote the identifier-term group of local unit r . Then

$$\begin{aligned}\text{ITID}^{(r)} &= \frac{|\{t \in G_r : t \in D\}|}{\max(1, |G_r|)}, \\ \text{NMI}^{(r)} &= \sum_{t \in G_r} \text{depth}(t), \\ \text{NM}^{(r)} &= \frac{1}{\max(1, |G_r|)} \sum_{t \in G_r} \text{senses}(t),\end{aligned}\quad (\text{A.5})$$

where D is the WordNet dictionary, $\text{depth}(t)$ is the maximum hypernym depth of token t , and $\text{senses}(t)$ is the number of WordNet senses of t . Snippet-level features are obtained by aggregating the local values across all valid local units. In particular, ITID and NMI use $\min(\cdot)$, $\text{avg}(\cdot)$, and $\max(\cdot)$, while NM uses $\text{avg}(\cdot)$ and $\max(\cdot)$.

CR. The implementation concatenates all comment text in s into a single sequence. Let $W(s)$, $S(s)$, and $Y(s)$ denote the resulting numbers of words, sentences, and syllables. Then

$$\text{CR}(s) = 206.835 - 1.015 \frac{W(s)}{S(s)} - 84.6 \frac{Y(s)}{W(s)}. \quad (\text{A.6})$$

TC. Let $\{b_1, \dots, b_q\}$ be the local textual blocks extracted from the snippet, where each block is a contiguous group of lines produced by the block-extraction heuristic in the implementation. For any pair (b_i, b_j) with $1 \leq i < j \leq q$,

$$\text{TC}^{(i,j)} = \frac{|T(b_i) \cap T(b_j)|}{|T(b_i) \cup T(b_j)|}. \quad (\text{A.7})$$

Snippet-level TC features are obtained by applying $\min(\cdot)$, $\text{avg}(\cdot)$, and $\max(\cdot)$ over all pairwise values $\{\text{TC}^{(i,j)}\}$.

NOC and NOC_norm. Each valid extracted line is represented as a token-set vector and clustered by DBSCAN. Let $C(s)$ be the set of non-noise clusters. Then

$$\begin{aligned}\text{NOC}(s) &= |C(s)|, \\ \text{NOC}_{\text{norm}}(s) &= \frac{|C(s)|}{m_{\text{val}}},\end{aligned}\quad (\text{A.8})$$

where m_{val} is the number of valid extracted lines used in clustering.

A.1.2 BWF. BWF consists of lightweight formatting and low-level structural statistics computed from lines, identifiers, and simple lexical counts. These features are implementation-aligned scalar statistics rather than outputs of a single unified counting formalism.

Let $L_{\text{code}}(s)$ denote the set of non-empty code lines in s , with $|L_{\text{code}}(s)| = m_{\text{code}}$. Representative features include average and maximum line length,

$$\begin{aligned}\text{LineLen}_{\text{avg}}(s) &= \frac{1}{\max(1, m_{\text{code}})} \sum_{l \in L_{\text{code}}(s)} |l|, \\ \text{LineLen}_{\text{max}}(s) &= \max_{l \in L_{\text{code}}(s)} |l|,\end{aligned}\quad (\text{A.9})$$

average and maximum indentation,

$$\begin{aligned}\text{Indent}_{\text{avg}}(s) &= \frac{1}{\max(1, m_{\text{code}})} \sum_{l \in L_{\text{code}}(s)} \text{indent}(l), \\ \text{Indent}_{\text{max}}(s) &= \max_{l \in L_{\text{code}}(s)} \text{indent}(l),\end{aligned}\quad (\text{A.10})$$

and average per-line counts of lexical categories,

$$\text{Count}_{\chi}(s) = \frac{1}{\max(1, m_{\text{code}})} \sum_{l \in L_{\text{code}}(s)} \#_{\chi}(l), \quad (\text{A.11})$$

where $\#_{\chi}(l)$ is the implementation-specific count of category χ on line l .

For identifiers, let $I(s)$ be the extracted identifier collection, where $|I(s)|$ counts multiplicity. Then

$$\begin{aligned}\text{IdLen}_{\text{avg}}(s) &= \frac{1}{\max(1, |I(s)|)} \sum_{t \in I(s)} |t|, \\ \text{IdLen}_{\text{max}}(s) &= \max_{t \in I(s)} |t|, \\ \text{CharPerLine}_{\text{max}}(s) &= \max_{l \in L_{\text{code}}(s)} |l|, \\ \text{IdPerLine}_{\text{max}}(s) &= \max_{l \in L_{\text{code}}(s)} |I(l)|.\end{aligned}\quad (\text{A.12})$$

Some BWF features are ratios, such as

$$\begin{aligned}\text{BlankLineRatio}(s) &= \frac{m_{\text{blank}}(s)}{\max(1, m)}, \\ \text{CommentLineRatio}(s) &= \frac{m_{\text{comment}}(s)}{\max(1, m)},\end{aligned}\quad (\text{A.13})$$

where $m_{\text{blank}}(s)$ and $m_{\text{comment}}(s)$ denote the numbers of blank lines and detected comment lines or comment segments.

A.1.3 PF. PF captures size-related and information-theoretic properties.

LOC. LOC is the number of non-empty lines:

$$\text{LOC}(s) = \sum_{i=1}^m \mathbf{1}[l_i \neq \emptyset]. \quad (\text{A.14})$$

Entropy. For line l_i , let $p_i(c)$ be the empirical probability of character c among all characters on that line. The implementation uses character-level entropy:

$$H(l_i) = - \sum_{c \in \mathcal{V}_{\text{char}}(l_i)} p_i(c) \log_2 p_i(c), \quad (\text{A.15})$$

where $\mathcal{V}_{\text{char}}(l_i)$ is the set of distinct characters on line l_i . At the snippet level,

$$\begin{aligned}\text{Entropy}_{\text{avg}}(s) &= \text{avg}(\{H(l_i) : l_i \neq \emptyset\}), \\ \text{Entropy}_{\text{std}}(s) &= \text{std}(\{H(l_i) : l_i \neq \emptyset\}).\end{aligned}\quad (\text{A.16})$$

We further define

$$\mathcal{V}_{\text{char}}(s) = \bigcup_{i=1}^m \mathcal{V}_{\text{char}}(l_i). \quad (\text{A.17})$$

Halstead Volume. Let η_1, η_2 be the numbers of distinct operators and operands, and let N_1, N_2 be their total occurrences. With $\eta = \eta_1 + \eta_2$ and $N = N_1 + N_2$,

$$\text{HV}(s) = N \log_2 \eta. \quad (\text{A.18})$$

A.1.4 DF. DF models the snippet as a visual object using lightweight display-oriented approximations. These features should be interpreted as heuristic proxies, not exact geometric measures.

Let

$$N_{\text{char}}(s) = \sum_{i=1}^m |l_i|, \quad (\text{A.19})$$

and let $N_{\text{kw}}(s)$, $N_{\text{str}}(s)$, and $N_{\text{com}}(s)$ denote the numbers of keyword, string-literal, and comment characters identified by the implementation. Then

$$\begin{aligned} \text{VKD}(s) &= \frac{N_{\text{kw}}(s)}{\max(1, N_{\text{char}}(s))}, \\ \text{VSD}(s) &= \frac{N_{\text{str}}(s)}{\max(1, N_{\text{char}}(s))}, \\ \text{VCD}(s) &= \frac{N_{\text{com}}(s)}{\max(1, N_{\text{char}}(s))}. \end{aligned} \quad (\text{A.20})$$

Visual complexity is approximated by the normalized size of the character set:

$$\text{VC}(s) = \frac{|\mathcal{V}_{\text{char}}(s)|}{95}. \quad (\text{A.21})$$

Let

$$W_{\text{max}}(s) = \max_{1 \leq i \leq m} |l_i|. \quad (\text{A.22})$$

Then the spatial features are

$$\begin{aligned} \text{SAA}(s) &= N_{\text{char}}(s), \\ \text{SRA}(s) &= \frac{N_{\text{char}}(s)}{\max(1, m \cdot W_{\text{max}}(s))}, \\ \text{SD}(s) &= \frac{m_{\text{ne}}}{\max(1, m)}, \\ \text{SR}(s) &= 1 - \frac{\text{std}(\{|l_i|\}_{i=1}^m)}{\max(1, \text{avg}(\{|l_i|\}_{i=1}^m))}. \end{aligned} \quad (\text{A.23})$$

Let $P_{\text{asg}}(s)$ and $P_{\text{br}}(s)$ denote the collections of column positions of assignment operators and bracket symbols, respectively. We use the convention $\text{std}(P) = 0$ if $|P| \leq 1$. Then

$$\begin{aligned} \text{AlignOp}(s) &= \frac{1}{1 + \text{std}(P_{\text{asg}}(s))}, \\ \text{AlignBr}(s) &= \frac{1}{1 + \text{std}(P_{\text{br}}(s))}, \\ \text{AlignCons}(s) &= \frac{\text{AlignOp}(s) + \text{AlignBr}(s)}{2}. \end{aligned} \quad (\text{A.24})$$

Let $I_{\text{user}}(s)$ denote the extracted user-defined identifier collection, and let $I_{\text{eng}}(s) \subseteq I_{\text{user}}(s)$ denote the subset whose decomposed terms match English words. Then

$$\text{TextEnglish}(s) = \frac{|I_{\text{eng}}(s)|}{\max(1, |I_{\text{user}}(s)|)}. \quad (\text{A.25})$$

Let $W_{\text{com}}(s)$ be the number of comment words and $W_{\text{code}}(s)$ the number of identifier-derived code words. Then

$$\text{TextComment}(s) = \frac{W_{\text{com}}(s)}{\max(1, W_{\text{com}}(s) + W_{\text{code}}(s))}. \quad (\text{A.26})$$

The remaining text-based display measures are

$$\begin{aligned} \text{TextVocab}(s) &= |\mathcal{V}_{\text{id}}(s)|, \\ \text{TextIdLen}(s) &= \frac{1}{\max(1, |I_{\text{user}}(s)|)} \sum_{t \in I_{\text{user}}(s)} |t|, \end{aligned} \quad (\text{A.27})$$

where $\mathcal{V}_{\text{id}}(s)$ is the set of unique extracted user-defined identifiers.

A.2 Extraction rules and preprocessing details

We apply a unified snippet-level workflow to all samples. To preserve structural and visual signals, preprocessing is limited to lightweight normalization, including line-ending standardization, removal of obvious encoding artifacts, and trimming of superfluous leading and trailing blank lines, while preserving internal white-space, indentation, blank lines, and comment placement.

After preprocessing, each snippet is processed with lightweight language-aware extraction rules based on lexical scanning and regular-expression matching. We identify identifiers, comments, keywords, operators, delimiters, literals, and line boundaries without relying on full parsing, which keeps the pipeline applicable to incomplete or noisy snippets.

WordNet is used for synonym expansion, sense counting, and semantic-depth estimation. Identifier tokens are normalized by lowercasing and decomposing naming conventions such as camelCase and snake_case. For a token not found in WordNet, its synonym set is empty, with sense- and depth-based quantities set to zero.

For NOC estimation, each valid line is represented as a token-set vector. For two lines l_i and l_j , the token-overlap distance is defined as

$$\delta(l_i, l_j) = 1 - \frac{|T(l_i) \cap T(l_j)|}{|T(l_i) \cup T(l_j)|}. \quad (\text{A.28})$$

DBSCAN is then applied to obtain the number of non-noise clusters.

After extraction, all quantities are represented as snippet-level scalar features and concatenated in a fixed order:

$$\mathbf{x} = [\text{TF}, \text{BWF}, \text{PF}, \text{DF}] \in \mathbb{R}^{61}. \quad (\text{A.29})$$

If a metric is undefined because the required evidence is absent, we assign a default value of zero to keep the feature dimension fixed.

Once the raw feature vector $\mathbf{x}$ is extracted, we apply feature-wise z-score normalization:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}, \quad (\text{A.30})$$

where μ_j and σ_j are the empirical mean and standard deviation estimated by the scaler used in the training pipeline.

The feature weights are obtained from the coefficient vector $\mathbf{w}$ of a Logistic Regression model fitted on the training-set features after StandardScaler normalization and SFS-based subset selection, and are used as weights to compute the readability score

$$\sum_j x'_j w_j. \quad (\text{A.31})$$

In addition, we formulate readability assessment as a supervised learning task using Logistic Regression. Given the normalized feature vector $\mathbf{x}'$, the model predicts

$$p(y = 1 | \mathbf{x}') = \sigma(\mathbf{w}^\top \mathbf{x}' + b) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x}' + b))}, \quad (\text{A.32})$$

where $\mathbf{w}$ is the coefficient vector, b is the intercept, and $\sigma(\cdot)$ is the logistic sigmoid. We use L2 regularization.

To reduce redundancy, we further apply Sequential Forward Selection (SFS). Let $\mathcal{F}$ denote the full candidate feature set and $\mathcal{S}_t$ the selected subset after step t . The procedure starts from

$$\mathcal{S}_0 = \emptyset. \quad (\text{A.33})$$

At step $t + 1$, we add the feature maximizing the model-selection criterion:

$$\begin{aligned} f^\star &= \arg \max_{f \in \mathcal{F} \setminus \mathcal{S}_t} \mathcal{M}(\mathcal{S}_t \cup \{f\}), \\ \mathcal{S}_{t+1} &= \mathcal{S}_t \cup \{f^\star\}. \end{aligned} \tag{A.34}$$

Moreover, at inference time, the optimized model outputs

$$r(s) = p(y = 1 \mid \mathbf{x}'(s)) = \sigma(\mathbf{w}^\top \mathbf{x}'(s) + b), \tag{A.35}$$

which is used as the final readability prediction. In addition to the full-model prediction rate, we also compute family-specific prediction rate by fitting separate Logistic Regression models on TF, BWF, PF, and DF.